

Week 2 Day 2

Task 1: LangChain setup and core concepts

The first task introduces the main LangChain pieces and connects them to the raw Python approach used in Day 1. ChatGoogleGenerativeAI acts as the LangChain wrapper around Gemini. Tools are defined with the `@tool` decorator, while `create_tool_calling_agent` and `AgentExecutor` handle the agent loop. `RunnableWithMessageHistory` is used later to provide conversation memory.

The task also builds a basic LCEL pipeline: `prompt` → Gemini model → output parser. The pipe operator connects these Runnable components so the output from one becomes the input to the next. The chain is assembled first and actually runs when `invoke`, `stream`, or `batch` is called.

Task 2: Define and register tools

Three tools are registered. The calculator evaluates basic arithmetic, `get_weather` provides the simulated weather data used in Day 1, and `get_product_price` is a new tool that reads a local products CSV file and returns price and stock information.

Each tool has a docstring describing what it does. LangChain uses the function signature, type hints, and docstring to create the tool schema and description that Gemini can use when deciding which tool is appropriate. This makes clear tool descriptions important for correct tool selection.

Task 3: Build the LangChain agent

The agent is created with `create_tool_calling_agent` and run through `AgentExecutor`. `AgentExecutor` is configured with verbose output, intermediate steps, a maximum number of iterations, and parsing-error handling. A multi-step example asks for an arithmetic result and Lahore weather, so the agent can select the calculator and weather tools during the same run.

The annotated trace can be understood as follows: Reason means Gemini decides what action is needed; Act is the tool invocation and its arguments; Observe is the result returned by the tool; Repeat/final means the executor continues the loop until Gemini gives the final answer. The internal reasoning text is not exposed by the trace. The important comparison with Day 1 is that the same Reason -> Act -> Observe -> repeat structure exists, but LangChain handles most of the orchestration instead of requiring a hand-written loop.

Task 4: Add memory

Task 4 adds `RunnableWithMessageHistory` so the agent can remember previous turns within a session. The notebook uses a session ID to keep a separate conversation history. The three-turn example first finds the Wireless Mouse price, then asks to compare it with the Mechanical Keyboard, and finally asks which one is better for a budget-conscious client.

The important result is that the later questions do not need to repeat the product names. LangChain automatically adds the earlier conversation to the agent prompt. This is the framework equivalent of manually maintaining the conversation contents list in Day 1.

Task 5: Structured output and error handling

The structured-output part defines a Pydantic model containing the cheaper item, the price difference, and a recommendation. A small second chain takes the agent's final answer and converts it into that structured model. This provides predictable fields instead of relying only on free-form text.

The error-handling example uses a tool that can simulate an intermittent catalog failure. `ToolException` and `handle_tool_error` turn the failure into a tool observation instead of letting the exception crash the agent. This is similar to the `try/except` approach from Day 1, but LangChain provides the mechanism through the tool configuration.

Raw Python vs. LangChain agent

In Day 1, the Gemini client, tool definitions, conversation history, and agent loop were controlled directly in Python. This gives more direct control and makes every step visible, but it also requires more code for orchestration and error handling.

In Day 2, LangChain wraps these pieces into reusable components. Tools can be registered with `@tool`, `AgentExecutor` manages the tool-calling loop, and `RunnableWithMessageHistory` manages session-based memory. The main advantage is less plumbing. The trade-off is that more framework behavior must be understood when debugging, especially when the Gemini integration exposes provider-specific message metadata.

Annotated reasoning trace

Example task: "What is 25 multiplied by 8, and what's the weather like in Lahore?"

Reason: Gemini identifies that two pieces of information are needed and selects the appropriate tools.

Act: The trace shows the selected tool being invoked with its arguments, such as calculator with the arithmetic expression and get_weather with Lahore.

Observe: Each tool returns its result to the agent. The calculator returns 200, while the weather tool returns the simulated Lahore weather.

Repeat/final: AgentExecutor gives the observations back to Gemini. Gemini uses them to produce the final response. This is the same basic agent pattern demonstrated manually in Day 1, with LangChain taking care of the loop.